

Folge 9: Was haben wir gelernt?

- Warum ein Netz zum Lernen eine "weiche" Aktivierung braucht statt eines harten Ein/Aus-Schwellenwerts (Sigmoid statt Stufenfunktion)
- Was ein Bias ist: ein freier Zahlenwert pro Neuron, unabhängig von den Eingaben, der die Entscheidungsgrenze verschieben kann
- Die Vorwärtsrichtung: Eingabe läuft wie gewohnt durch beide Schichten, jetzt inklusive Bias
- Die Rückwärtsrichtung (Backpropagation): der Fehler wird von der Ausgabe zurück durch das Netz geschoben, damit auch die versteckte Schicht weiß, wie sie sich anpassen soll
- Wie man mit NumPy ein komplettes Training über tausende Durchläufe kompakt schreibt
- Dass der durchschnittliche Fehler über die Zeit sichtbar kleiner wird, ein direkter Beweis, dass das Netz wirklich lernt

Folge 9: Fazit

- Der Bogen aus Folge 6 und Folge 7 schließt sich hier endgültig: das Problem, das ein einzelnes Neuron nicht lösen konnte, wird jetzt von einem Netz gelöst, das seine eigenen Gewichte komplett selbst gefunden hat
- Backpropagation klingt kompliziert, ist im Kern aber genau dieselbe Idee wie in Folge 7: Fehler anschauen, ein kleines Stück in die richtige Richtung korrigieren, nur eben Schicht für Schicht rückwärts durchgereicht
- Ohne Bias kann ein Netz bei XOR hängen bleiben, weil die Entscheidungsgrenze sonst zwingend durch den Nullpunkt verlaufen muss, ein kleines, aber entscheidendes Detail
- Das ist im Grunde derselbe Trainingsmechanismus, mit dem auch riesige Modelle wie ChatGPT trainiert werden, nur mit Millionen statt vier Trainingsbeispielen und Milliarden statt sechs Gewichten
- Damit ist unsere "Neuronales Netz von Grund auf"-Reise fürs Erste abgeschlossen, ab der dieser Folge wechseln wir die Algorithmen-Familie